Kafka Fundamentals for Java Developers

Section 1: Introduction

Introduction

1. Fundamentals:
    1. What is kafka
    2. Why do we need it
    3. Use-cases where it is used
    4. Who is using it
2. Kafka architectural components
    1. Records
    2. Topics
    3. Partitions
    4. Consumer groups
    5. ...
3. Kafka broker
4. Kafka cli
    1. Creation of brokers
    2. Send and receive messages
5. Kafka client API
    1. For producers and consumers
6. Simple and custom serializers and deserializers
    1. Object type records or events
7. Avro: Framework to send and receive data in a language neutral fashion
    1. Confluent schema registry
    2. Avro schema
    3. Generate stubs from schema (maven avro plugin)
8. Producers
    1. Partitioner classes
    2. Advanced properties
9. Consumers
    1. Consumer groups
    2. Rebalancing
    3. Offset commits
    4. Advanced properties
10. Stream processing
    1. Kafka streaming API
    2. Applications
11. Spring and Spring Boot with Kafka

How to make the best of this course

1. Lectures
2. Quiz
3. Assignments
4. Pre-requisites
    1. Software Setup
    2. IntelliJ or STS
5. Kafka setup
6. Kafka CLI
7. Confluent Platform

Completed projects on GitHub

1. github.com/bharaththippireddy

Slides Used in the Course

Download Assignment Solutions

Section 2: The Fundamentals

What is Kafka

1. Kafka is a distributed commit log
    1. Microservice applications put events onto a log
        1. Term for log is topic
    2. Kafka stores events in orderly fashion and writes events to disk (replicated disks - ensure that events are not lost)
2. Microservices exchange messages through topics or streams in realtime
    1. We can process data as soon as they are produced, we can have real-time analytics
        1. We can give recommendations based on the analytics
    2. Each service can define it's own computational logic
        1. Solution: **Streaming API** (the following operations can be done)
            1. Grouping
            2. Aggregating
            3. Filtering
            4. Joining
3. Data in external services/data bases
    1. Solution: Kafka connect
        1. Open source and commercial Kafka connectors (to fetch data into Kafka or send data to other sources)

Why Kafka

1. Advantages
    1. Multiple producers and consumers
        1. Multiple producers can write to a **single** topic at the same time
        2. Multiple consumers can subscribe and consume messages from the same topic
        3. Kafka retains message and multiple consumers can read the message and process at any time
        4. Kafka supports consumer groups and partitions
            1. We can have parallel consumers running which can be part of a consumer group
        5. Topic is divided into multiple partitions
            1. Messages from producers will go across the partitions to enable parallel processing
            2. Kafka ensures that within a consumer group, the message is **consumed only once**
                1. Only one consumer in the group will be able to consume a particular message
                    1. No duplication
            3. Enables multiple applications to process the same message but within an application, parallelism is allowed through consumer group
        6. Disk Based Persistence
            1. If a consumer is down due to restart or a crash, when it comes up, Kafka would have retained the message in a disk storage for configurable amount of time
        7. Scalability
            1. As the load on Kafka broker increases, the brokers can be easily scaled
                1. We can have **multiple brokers** (in cluster mode)
                    1. If one broker goes down, another broker will take over (availability)
        8. Performance
            1. Through the above mechanism

Usecases

1. Pub-Sub
    1. Used to exchange messages across micro-services
        1. One micro-service can send notification to a kafka topic and another micro-service will subscribe and consume the notification
    2. Producer-Consumer pattern can be implemented
        1. Order processing
        2. Shipment
        3. ...
2. Activity Tracking
    1. Used to track user activity and fed to a topic
    2. The events are read by an ML algorithm which can be used to give recommendations to the user
    3. Browser application can read the recommendations and push them to the user
        1. Example: Amazon, Netflix, ...
3. Metrics and Log Aggregation
    1. As applications produce metrics on a regular basis, all of them can be aggregated and stored in a **permanent storage** to do analysis
        1. Real-time
            1. Permanent storage:
                1. Elasticsearch
                2. ...
        2. Analysis
            1. Security threats
            2. Unwanted activities
4. Commit Log
    1. As and when DB changes happen, the changes can be streamed onto kafka topic
        1. Used to replicate database if required
        2. Used to analyse what is going on with the DB
            1. To check threads of unwanted activity
5. Stream Processing (custom use cases are supported)
    1. **Data Pipelines**
        1. As data flows through different stages of the pipeline, we can have transformations and computational logic (at each step)
            1. Big data tools like hadoop, storm, ... can be part of the stream processing

Where is Kafka Being Used

1. Data platforms
2. Micro-service applications
3. Monitoring Trucks, Cars, ... (in realtime)
4. Factories
    1. To analyse IoT device data - to make decisions on the fly
5. Financial transactions
    1. To process banking and stock exchange transactions
6. Hospitals
    1. Patient info can be tracked and monitored (any changes in condition can also be tracked to treat him better)
7. Retail, Hotel, and Travel
    1. Kafka can be used to gather and react to customer interactions and recommendations can be shown

Who is Using Kafka

1. Twitter
    1. Mobile app performance management and analytics
        1. 5 Billion sessions/day
2. Uber
    1. 1 Trillion events per day
        1. Log aggregation
        2. DB change
        3. Log maintenance
        4. Other even streaming use-cases
    2. uReplicator (contribution)

        1. Cluster replication solution to the open-source world
    3. Netflix
        1. Messaging backbone
            1. 4000 kafka brokers in cloud handling 7 Billion events per day
    4. Yahoo
        1. Real-Time analytics
            1. 20 GB data/second
    5. Pinterest
        1. Real Time Ads
            1. 2k brokers - handling 8B events per day

Kafka Architectural Components

1. Kafka Architecture
    1. Broker
        1. Kafka **cluster** is a collection of **Kafka brokers** (aka **Kafka servers** or **Kafka nodes**)
        2. **It is used to exchange messages between producer and consumer**
        3. Broker ensures that
            1. Producer and consumer are decoupled
            2. Messages are persisted and durable
        4. Broker is a **Java process**
        5. We can increase the brokers to provide scalability and durability of messages
        6. **One of the brokers will be elected as cluster leader or cluster controller**
            1. Other brokers will follow the leader
            2. **Leader**
                1. Responsible for managing
                    1. Partitions
                    2. Replicas
                    3. Other administrative operations
    2. ZooKeeper
        1. Responsible for electing a **cluster controller** or **leader**
            1. All broker nodes will register with ZooKeeper component (when they come up)
            2. Zookeeper will elect one of the brokers as leader
                1. If that leader goes down, it will pick up another broker as the leader
        2. Maintain state of the cluster (metadata of leader and followers)
            1. States
            2. Quotas
            3. Access Control List
        3. ZooKeeper is an Apache project of its own
            1. **Kafka comes bundled with ZooKeeper**
                1. Can get replaced with Kafka's own component in the future
    3. Producers
        1. Produce data
        2. Communicate using TCP protocol
        3. They connect directly with the brokers and start sending messages to topics
        4. **Producer can send messages to multiple topics and a topic can receive messages from multiple producers**
    4. Consumers
        1. Consumes from one or more topics (data or records) and processes
        2. **They coordinate among themselves to balance the load**
        3. **They track each other's progress**

Kafka Records

1. Record
    1. Producer constructs and exchanges data using **record**
    2. 7 attributes of record
        1. Topic
            1. Topic to which the record needs to be written to
        2. Partition
            1. 0 based index to which the record should be written
                1. **A record is associated only with one partition**
            2. Number can be set by producer
                1. If not set, the number is calculated based on **Key** the producer provides
                    1. Hash value is calculated and result is used to identify the partition
        3. Offset
            1. 64 bit signed integer
            2. Used to locate the record within a partition
        4. Timestamp
            1. Set by producer or if not set, producer api sets it internally to current time
        5. Key
            1. Optional non-unique value
            2. Array of bytes
                1. It is used to calculate partition number
            3. If this value is not set, Kafka will decide partition number in a round robin fashion (can change in the future)
            4. **If same key is set for multiple records, all those records will end up in the same partition**
                1. **To uniquely identify a record, use the following as composite key:**
                    1. **partition number**
                    2. **offset**
        6. Headers
            1. Optional key-value pair
                1. Just like HTTP headers
            2. Used to pass meta-data
        7. Value
            1. Payload
            2. Array of bytes
                1. Contains business data

3. It is optional (but no value makes no sense)

2. Assignment:
    1. Search for Kafka producer record
        1. https://kafka.apache.org/23/javadoc/org/apache/kafka/clients/producer/ProducerRecord.html

Topics Partitions Offsets and Replication

1. Topic
    1. Messages are sent to a topic
    2. Each topic can be divided into one or more partitions
        1. Partition is a single log of records
    3. Messages are appended to the end of a partition as they come in
    4. Each partition gets a unique number
    5. Each message inside a partition gets an offset value
        1. It is like array index and starts from 0
            1. Partition number + offset => uniquely identifies a record
        2. Kafka stores partition & offset values inside a record or message
    6. **Messaging order across partitions is not guaranteed but order within a partition is maintained**
    7. If producer specifies partition number, Kafka puts that message into that partition with that partition number
    8. If producer does not specify partition number, Kafka will calculate a hash if key is specified, that is the partition number
    9. If producer does not specify a partition number or key, Kafka puts the message into one of the partitions as per round robin algorithm
    10. Partitions give the property of scalability & availability
        1. The partitions can be scaled across brokers
            1. **Instead of putting all partitions in one broker, we can scale them across brokers**
    11. Partitions support replication/duplication
        1. Gives high availability
        2. Example:
            1. Partition 0 - can be present in broker 0 and broker 1
                1. If one broker goes down, another broker will take over
                    1. Which broker processes?
                        1. Leader & follower decide
                            1. Each partition has a leader and a follower
                                1. Broker 0
                                    1. Partition 0 (L)
                                    2. Partition 1 (L)
                                    3. Partition 3 (F)
                                2. Broker 1
                                    1. Partition 2 (L)
                                    2. Partition 0 (F)
                                    3. Partition 1 (F)
                                3. Broker 2
                                    1. Partition 3 (L)
                                    2. Partition 2 (F)
                            2. Only if **leader goes down**, **follower will take over**
                            3. **We can specify replication using the replication factor we want**
                                1. 1 - no duplication
                                2. 2 - one L, one F
                                3. 3 - one L, two F
                            4. Replication factor configuration:
                                1. Can be specified when defining a topic
                                2. Can be specified when constructing a cluster

Consumer Groups

1. Consumer group
    1. It is a set of consumers working together to consume a topic
    2. It ensures that **each partition is consumed by only one consumer**
        1. A consumer can consume from multiple partitions as well
    3. Assigning a consumer to a partition is called the ownership of partition by consumer
    4. **Consumers can be horizontally scaled**
        1. If load on a consumer is increasing, a consumer can be created which will take over a partition
    5. If a consumer fails:
        1. Remaining consumers can coordinate among themselves and take over the partitions that the failed consumer was working with

Batching

1. KafkaProducer:
    1. Does not send only one message to the broker

        1. They batch them to the topic and partition that they have to go to

```
Topic 1
Partition 0
- Batch 0
- Batch 1
- Batch 2

Topic 2
Partition 1
- Batch 0
- Batch 1
- Batch 2
```

        2. **Batch**

            1. It is a collection of **messages** all of which must be written to the **same topic** and **same partition**
            2. Reduces network round-trips (by avoiding sending each message across network to broker)

        1. Larger the batch size, more messages can be processed in a given timeframe
    3. Each batch is **compressed** providing more efficient data transfer and storage
        1. Trade-off: processing time
    4. Trade-off:
        1. If batch size is too big, there might be more delay before sending the message
        2. If batch size is too small, there are more network round-trips

Quiz 1: The Fundamentals

Section 3: Kafka in Action

Introduction

1. Kafka up and running on local machine (Windows and Mac)
2. Kafka cli to construct and manage kafka topics
3. Kafka producers and consumers

Java Version

1. Kafka is happy usually happy with any Java version
2. Second method:
    1. Download Confluent Platform & launch Kafka
        1. Launches several other useful tools like **Schema Registry**
            1. Confluent needs Java 8 and Java 11 (not higher version)
                1. Code can be written in a higher version of Java
                    1. Don't add higher version of Java to path for Confluence

Install on Windows

1. `java -version`
2. Search for **kafka download**
    1. Download binary latest version
        1. `.tgz` file (windows and linux)
    2. `bin`
        1. All binaries
            1. `.sh` - linux
            2. `windows` - windows
                1. zookeeper server start and stop
                2. kafka server start and stop
        2. Add `bin\windows` to PATH in Windows
    3. `config`
        1. Zookeeper properties
        2. Kafka properties
3. Open new CMD
    1. Start zookeeper:

```
zookeeper-server-start C:\<path>\config\zookeeper.properties
```

4. Open new CMD
    1. Start kafka broker:

```
kafka-server-start C:\<path>\config\server.properties
```

Install on Mac

1. Using homebrew

```
brew install kafka
```

    1. cmd + shift + g (from terminal)
        1. Copy `/usr/local/Cellar/kafka/2.8.0/`
            1. Has binaries
    2. `/usr/local/etc/kafka`
        1. Has configuration files
            1. Kafka assumes zookeeper is running on localhost:2181
2. Start zookeeper

```
zookeeper-server-start /usr/local/etc/kafka/zookeeper.properties
```

3. Start kafka

```
kafka-server-start /usr/local/etc/kafka/server.properties
```

Kafka Commandline

1. Using kafka commandline

2. Download txt from resources

3. Commands:

```
kafka-topics --list --bootstrap-server localhost:9092 # **(M)**

# shows default topic - __consumer_offsets (used to store offset details)

kafka-topics --create --replication-factor 1 --partitions 1 --topic first-topic --bootstrap-server localhost:9092

# Run the list command to see the new topic

kafka-topics --describe --topic first-topic --bootstrap-server localhost:9092

# Topic name, id, config info, replicas, replication factor, ...

kafka-console-consumer --bootstrap-server localhost:9092 --topic first-topic

# starts polling the topic

kafka-console-producer --broker-list localhost:9092 --topic first-topic

> kafka is easy

# consumer displays the message

# ctrl + c to stop

kafka-topics --delete --bootstrap-server localhost:9092 --topic first-topic

# list the topics and you don't see the deleted topic
# If delete is not working set the following in server.properties

delete.topic.enable=true
```

Kafka APIs

1. Kafka gives 5 APIs (in Java and Scala)
    1. Python also has support
2. APIs
    1. Admin API
        1. Used to manage
            1. Topics
            2. Brokers
            3. Other objects
        2. KafkaDrop: GUI based tool makes use of Admin API and give us browser based access to Kafka cluster
    2. Producer API and Consumer API
        1. Producer API: Used to publish stream of
            1. Events
            2. Records
            3. Messages
        2. Consumer API: Used to read the messages
        3. Producer API
            1. They take care of:
                1. Serialization
                2. Partition assignment
                3. ...
        4. Consumer API
            1. They take care of:
                1. DeSerialization
                2. Rebalancing
                3. ...
    3. Streams API
        1. Allows us to implement stream processing applications
            1. Allows to process event streams and apply transformations and maintain state...
        2. We read from a topic, apply tranformation and write to another topic
    4. Connect API
        1. To build and run re-usable data import and export connectors
        2. They allow us to import and export data from other systems to integrate with Kafka
            1. There are ready to use connectors
                1. MySQL
                2. PostgreSQL
                3. Elasticsearch
                4. ...
            2. The connectors are used to fetch data from the sources or push data to the sources

Documentation

1. To learn more:
    1. Search for kafka documentation
        1. Apache kafka website
            1. Read the docs
                1. Examples
        2. Confluent Documentation
            1. Tutorials
            2. It is a cloud based implementation of Apache opensource Kafka
                1. Comes with ecosystem of products (not just Kafka)
                    1. Confluent CLI
                    2. KsqlDB
                    3. Schema Registry
            3. Also gives platform to run locally or on a server

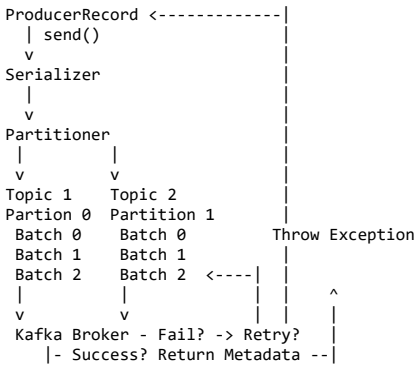
4. Click on Clients
    1. Producer and Consumer APIs
    2. More examples and wrappers
2. Search for **confluent kafka tutorial**

Quiz 2: Kafka in action

Section 4: Construct Producers and Consumers

Kafka Producer

1. Workflow:



    1. Record:
        1. We can set topic & value
    2. Send() - Producer hands over record to a Serializer
        1. Serializer converts Java type into byte array (using **key** and **value**)
            1. Kafka has inbuilt serializers that can work with common types
                1. Custom serializers can be written
                    1. Avro can be used as well
    3. Serializer hands the record over to a partitioner
        1. Partitioner checks if the record has a partition number
        2. If partition number exists, it will use it, or else, it will use value of key and hashing algorithm to calculate partition number. If both don't exist, it will assign using round robin
        3. Partitioner knows to which partition the record must go to
            1. It adds the record to a batch of a particular topic and partition
    4. A separate thread picks up batch and sends to kafka broker
    5. If broker receives the message and writes it to Kafka, we get **record metadata** back
    6. If write fails, we get failure back
        1. Producer can retry again to send back to that partition. If it fails after retry, it will throw an exception

Producer API Walkthrough

1. Construct an instance of `KafkaProducer<String, Integer>` class (constructor takes properties)
    1. Key
    2. Value
2. `Properties`
    1. Mandatory properties

```
props.setProperty("bootstrap.servers", "localhost:9092"); // list of brokers, the producer should connect to
props.setProperty("key.serializer", "org.apache.kafka.common.serialization.StringSerializer"); // Serializer is used to convert simple typ
props.setProperty("value.serializer", "org.apache.kafka.common.serialization.IntegerSerializer"); // converts value into byte array

KafkaProducer<String, Integer> producer = new KafkaProducer<String, Integer>(props);
ProducerRecord<String, Integer> record = new ProducerRecord<>("OrderTopic", "Mac Book Pro", 10); // Topic, Key, Value

try {
    producer.send(record, new OrderCallback()); // fire and forget manner - don't handle response
} catch (Exception e) {
    e.printStackTrace();
} finally {
    producer.close();
}
```

    1. Serializers implement `Serializer` interface from Kafka
    2. To receive response:

```
Future<RecordMetadata> send = producer.send(record);
RecordMetadata recordMetadata = send.get(); // waits until response is obtained
```

    3. Asynchronous: Passing callback

Construct a Producer Project

1. Construct Maven project
    1. Search for: `archetype-quickstart`
        1. Group ID: org.bharath.kafka
        2. Artifact: orderproducer

    2. Remove the entire build section if it exists
    3. Specify Java version
2. Search for kafka clients maven dependency
    1. Latest version
        1. `kafka-clients`

Construct a Producer

1. OrderProducer class with main method

```
Properties props = new Properties();
props.setProperty("bootstrap.servers", "localhost:9092");
props.setProperty("key.serializer", "org.apache.kafka.common.serialization.StringSerializer"); // ctrl+shift+t
props.setProperty("value.serializer", "org.apache.kafka.common.serialization.IntegerSerializer"); // ctrl+shift+t

KafkaProducer<String, Integer> producer = new KafkaProducer<String, Integer>(props);
ProducerRecord<String, Integer> record = ProducerRecord<>("OrderTopic", "Mac Book Pro", 10);

try {
    producer.send(record);
    System.out.println("Message Sent Successfully");
} catch (Exception e) {
    e.printStackTrace();
} finally {
    producer.close();
}
```

Sync Send

1. `Future<RecordMetadata> future = producer.send(record);`

```
RecordMetadata recordMetadata = producer.send(record).get();
System.out.println(recordMetadata.partition());
System.out.println(recordMetadata.offset());
```

Async Send

1. Callback

```
producer.send(record, new OrderCallback());
```

    1. Callback

```
import org.apache.kafka.clients.producer.Callback;

public class OrderCallback implements Callback {
    @Override
    public void onCompletion(RecordMetadata metadata, Exception exception) {
        System.out.println(metadata.partition());
        System.out.println(metadata.offset());
        if (exception != null) {
            exception.printStackTrace();
        }
    }
}
```

API Walkthrough

1. OrderConsumer

```
Properties props = new Properties();
props.setProperty("bootstrap.servers", "localhost:9092");
props.setProperty("key.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
props.setProperty("value.deserializer", "org.apache.kafka.common.serialization.IntegerDeserializer");
props.setProperty("group.id", "OrderGroup"); // consumer belongs to a group (not producer)

KafkaConsumer<String, Integer> consumer = new KafkaConsumer<>(props);
consumer.subscribe(Collections.singletonList("OrderTopic")); // consumer can subscribe to multiple topics - even using regex

ConsumerRecords<String, Integer> orders = consumer.poll(Duration.ofSeconds(20)); // Consumer polls for messages with timeout
for (ConsumerRecord<String, Integer> order : orders) {
    System.out.println("Product Name " + order.key());
    System.out.println("Quantity " + order.value());
}
consumer.close();
```

    1. `poll` - does many things behind the scenes
        1. Partition re-balancing
        2. Checks heart beats
        3. Data fetching

Construct a Consumer Project

1. Spring Tool Suite > File Menu > Other > Maven Project > Next > Next > Search for "maven-archetype-quickstart"
2. Group id: com.bharath.kafka
3. Artifact id: order-consumer
4. Open pom.xml
    1. `maven-compiler-source`: `14`
    2. `maven-compiler-target`: `14`

5. Delete `build` section
6. Paste the following under `dependencies`

```
<dependency>
    <groupId>org.apache.kafka</groupId>
    <artifactId>kafka-clients</artifactId>
    <version>2.8.0</version>
</dependency>
```

1. Do **Maven > Update Project**

Consume Messages

1. New class: `OrderConsumer` with a `main` method:

```
public static void main(String[] args) {
    Properties props = new Properties();
    props.setProperty("bootstrap.servers", "localhost:9092");
    props.setProperty("key.deserializer", "org.apache.kafka.common.serialization.StringDeserializer);
    props.setProperty("value.deserializer", "org.apache.kafka.common.serialization.IntegerDeserializer");
    props.setProperty("group.id", "OrderGroup"); // group the consumer belongs to

    KafkaConsumer<String, Integer> consumer = new KafkaConsumer<>(props);
    consumer.subscribe(Collections.singletonList("OrderTopic"));

    // consumer.poll(Duration.ZERO); // polls to see if the topic has any messages, or else returns
    ConsumerRecords<String, Integer> orders = consumer.poll(Duration.ofSeconds(20)); // within 20 seconds checks if there is a message or else

    for (ConsumerRecord<String, Integer> order : orders) {
        System.out.println("Product Name" + order.key());
        System.out.println("Product Value" + order.value());
    }
    consumer.close();
}
```

Quiz 3: Construct Producers and Consumers

Assignment 1: Construct Consumers and Producers

1. Producer:

```
Properties props = new Properties();
props.setProperty("bootstrap.servers", "localhost:9092");
props.setProperty("key.serializer", "org.apache.kafka.common.serialization.StringSerializer"); // ctrl+shift+t
props.setProperty("value.serializer", "org.apache.kafka.common.serialization.StringSerializer"); // ctrl+shift+t

KafkaProducer<String, String> producer = new KafkaProducer<String, String>(props);
ProducerRecord<String, String> record = ProducerRecord<>("TruckTrackerTopic", "IER242323", "22.5726 N, 88.3639 E");

try {
    producer.send(record);
    System.out.println("Message Sent Successfully");
} catch (Exception e) {
    e.printStackTrace();
} finally {
    producer.close();
}
```

2. Consumer:

```
Properties props = new Properties();
props.setProperty("bootstrap.servers", "localhost:9092");
props.setProperty("key.deserializer", "org.apache.kafka.common.serialization.StringDeserializer);
props.setProperty("value.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
props.setProperty("group.id", "TruckTrackerGroup"); // group the consumer belongs to

KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props);
consumer.subscribe(Collections.singletonList("TruckTrackerTopic"));

ConsumerRecords<String, String> truckTrackers = consumer.poll(Duration.ofSeconds(20)); // within 20 seconds checks if there is a message or els

for (ConsumerRecord<String, Integer> truckTracker : truckTrackers) {
    System.out.println("Truck Tracker Id" + truckTracker.key());
    String[] latLong = truckTracker.split(",");
    System.out.println("Latitude: " + latLong[0]);
    System.out.println("Longitude: " + latLong[1]);
}
consumer.close();
```

Section 5: Use Custom Serializers and Deserializers

Introduction

1. In-built Kafka serializers and deserializers can be used to convert Strings, Integers, Longs, ... into byte arrays.
2. For custom object types, we need to define our own custom serializers and deserializers
3. Steps:
    1. Implement `Serializer<T>` from Kafka
    2. Provide implementation
        1. Convert object into a byte array
            1. An easy way: Jackon library (object mapper, convert to a string and then into bytes)
    3. Implement `Deserializer<T>` from Kafka
    4. Provide implementation
        1. Convert a byte array into an object
            1. An easy way: Jackson library (object mapper' `readValue`)
    5. Use the serializer and deserializer

1. Provide serializer and deserializer as properties

```
props.setProperty("value.serializer", "<package>.OrderSerializer");
...
props.setProperty("value.deserializer", "<package>.OrderDeserializer");
```

Construct a Serilizer

1. Order class:

```
public class Order {
    private String customerName;
    private String product;
    private int quantity;

    // Getter and setters
}
```

2. OrderSerializer extends org.apache.kafka.common.serialization.Serializer

```
public class OrderSerializer extends Serializer<Order> {
    @Override
    public byte[] serialize(String topic, Order order) {
        byte[] response;
        ObjectMapper objectMapper = new ObjectMapper();
        try {
            response = objectMapper.writeValueAsString(order).getBytes();
        } catch (JsonProcessingException e) {
            e.printStackTrace();
        }
        return response;
    }
}
```

    1. Add **com.fasterxml.jackson.core:jackson-databind:2.12.4** to pom.xml

3. OrderProducer

```
...
props.setProperty("value.serializer", "com.bharath.kafka.orderproducer.customserializers.OrderSerializer");

KafkaProducer<String, Order> producer = new KafkaProducer<>(props);
Order order = new Order();
order.setCustomerName("Bharath");
order.setProduct("IPhone");
order.setQuantity(1);
ProducerRecord<String, Order> record = new ProducerRecord<>("OrderCSTopic", order.getCustomerName(), order); // keep topics clean by keeping th
```

Construct a Deserializer

1. Add Jackson dependency in consumer project (after `kafka-client` dependency)

2. New Package: `customerdeserializer`

3. OrderDeserializer implements org.apache.kafka.common.serialization.Deerializer

```
@Override
public Order deserialize(String topic, byte[] data) {
    ObjectMapper objectMapper = new ObjectMapper();
    Order order = null;
    try {
        order = objectMapper.readValue(data, Order.class);
    } catch (IOException e) {
        e.printStackTrace();
    }
    return order;
}
```

4. Open `OrderConsumer`

```
props.setProperty("value.deserializer", OrderDeserializer.class.getName());
...
KafkaConsumer<String, Order> consumer = new KafkaConsumer<>(props);
consumer.subscribe(Collections.singletonList("OrderCSTopic"));

ConsumerRecords<String, Order> records = consumer.poll(Duration.ofSeconds(20));
for (ConsumerRecord<String, Order> record : records) {
    String customerName = record.key();
    Order order = record.value();
    System.out.println("Customer Name: " + customerName());
    System.out.println("Product: " + order.getProduct());
}
```

5. Run the applications as Java applications

Quiz 4: Use Custom Serializers and Deserializers

Assignment 2: Use Custom Serializers and Deserializers

1. TruckTracker

Section 6: Use Avro

Introduction

1. We do not have to define customer serializers and deserializers
2. Avro: Apache open-source project
    1. Gives us a language neutral syntax
        1. Schema file:
            1. Consumer side: Language object (Order, Consumer, Logs, ...)
            2. Producer side: `KafkaAvroSerializer`
                1. It knows how to serialize a language object into a byte array by looking at the schema file
            3. We then hand the byte array to Kafka broker
            4. Consumer side: `KafkaAvroDeserializer`
            5. Producer and consumer exchange the schema file through a **Schema Registry**
                1. `KafkaAvroSerializer` pushes schema to the **Schema Registry**
            6. `KafkaAvroDeserializer` will look at Consumer configuration where Schema Registry URL is defined and pull the schema
    2. Schema:
        1. JSON
            1. JSON String
            2. JSON Object (typically)
            3. JSON Array
        2. Data types used:
            1. null
            2. int
            3. long
            4. float
            5. double
            6. bytes
            7. string
        3. Example:

```
{
    "namespace": "com.bharath.kafka.avro",
    "type": "record",
    "name": "Order",
    "fields": [
        {
            "name": "customerName",
            "type": "string"
        },
        {
            "name": "product",
            "type": "string"
        },
        {
            "name": "quantity",
            "type": "int"
        }
    ]
}
```

            1. `type`: default value is `record`
        4. We can generate java objects using the schema

Schema Registry

1. Each record produced will comply to Avro schema
    1. Whenever schema changes, consumer needs to know about the change
        1. Hence the schema registry servers
    2. Schema registry servers must maintain different versions of the schema

The Behind the Scenes

1. Schema Registry:
    1. Producers:
        1. Avro Schema file
        2. `AvroSerializer` class
            1. Converts data into bytes
            2. Ensures that the data in compliant with the schema
            3. Pushes schema file to Schema registry
            4. Subject:
                1. It is a scope in which a schema evolves over time
            5. Stores unique `SchemaId` (assigned by Schema Registry) in Header section of record going out
            6. Puts data in record & record will be sent to Kafka brokers
    2. Consumer
        1. `AvroDeserializer` class
            1. Looks at `SchemaId` in the headers section of the record
            2. Connects to schema registry (using Schema Registry URL provided in the consumer)
            3. Fetches the schema
            4. Validates the data that came in against the schema
    3. Versioning
        1. As new schemas are pushed, the versioning in maintained b the schema registry
            1. Latest schema version id is maintained in the record that will be sent to the consumer
            2. Consumer validates the data as per the id that came in
    4. Schema registry stores the schemas behind the scenes in Kafka in `_schemas` topic

Launch Confluent Schema Registry

1. Two ways to get Schema Registry
    1. Search for **kafka schema registry github**
        1. confluent/schema-registry
            1. Download the code and follow the instructions
    2. Search for **confluent platform download**
        1. Click on **Free Confluent Trial...**
            1. What is confluent?
                1. It is a cloud based implementation of the entire Kafka ecosystem (a lot of tools)
        2. Two options:
            1. Cloud
            2. Software

                1. We can download the entire confluent platform locally

                    1. Unzip/install
                        1. bin/kafka*
                        2. bin/confluent*
                            1. windows/*.bat

                2. Add the folder upto bin/windows in PATH

                3. Add the folder **bin** using `export` commands in Mac and Linux

                ```
                export CONFLUENT_HOME=<confluent-home-directory>
                export PATH=$PATH:$CONFLUENT_HOME/bin
                ```

                4. Stop Kafka, Zookeeper etc...

                5. Run the following command to start confluent platform:

                ```
                confluent local services start # **(M)**
                ```

                    1. Starts:
                        1. Zookeeper
                        2. Kafka
                        3. Schema Registry
                        4. Kafka REST API (Connect)
                        5. ...

                6. Open Schema Registry:

                ```
                http://localhost:8081/schemas
                ```

Construct Avro Schema

1. New source folder: src/main/resources

    1. New file: order.avsc

    ```
    {
        "namespace": "com.bharath.kafka.avro",
        "type": "record",
        "name": "Order",
        "fields": [
            {"name": "customerName", "type": "string"},
            {"name": "product", "type": "string"},
            {"name": "quantity", "type": "int"}
        ]
    }
    ```

        1. namespace: used as package name
        2. name: used as class-name

API Walkthrough

1. Steps:

    1. Add Maven dependencies

    2. Configure Avro Maven plugin

        1. To generate Java class out of the schema file
            1. Order Pojo can be generated

    3. Producer creation

    ```
    props.setProperty("key.serializer", "io.confluent.kafka.serializers.KafkaAvroSerializer");
    props.setProperty("value.serializer", "io.confluent.kafka.serializers.KafkaAvroSerializer");
    props.setProperty("schema.registry.url", "http://localhost:8081");
    props.setProperty("specific.avro.reader", "true");
    props.setProperty("group.id", "AvroOrderGroup");

    KafkaProducer<String, Order> producer = new KafkaProducer<>(props);

    ProducerRecord<String, Order> record = new ProducerRecord<>("OrderTopic", order.getCutomerName().toString());

    try {
        producer.send(record);
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        producer.close();
    }
    ```

1. `KafkaAvroSerializer`:
    1. Works with common types (String, int, ...) & object types
    2. It also pushes schema to schema registry
2. `specific.avro.reader`: for specific objects (`Order`)

Configure Maven Avro Plugin

1. Dependencies:

```
<dependencies>
    <dependency>
        <groupId>org.apache.avro</groupId>
        <artifactId>avro</artifactId>
        <version>1.10.2</version>
    </dependency>
    <dependency>
        <groupId>io.confluent</groupId>
        <artifactId>kafka-avro-serializer</artifactId>
        <version>6.2.0</version>
    </dependency>
<dependencies>

<repositories>
    <repository>
        <id>confluent</id>
        <url>http://packages.confluent.io/maven/</url>
        <releases>
            <enabled>true</enabled>
        </releases>
        <snapshots>
            <enabled>true</enabled>
        </snapshots>
    </repository>
</repositories>

<build>
    <plugins>
        <plugin>
            <groupId>org.apache.avro</groupId>
            <artifactId>avro-maven-plugin</artifactId>
            <version>1.10.2</version>
            <executions>
                <execution>
                    <phase>generate-sources</phase>
                    <goals>
                        <goal>schema</goal>
                    </goals>
                    <configuration>
                        <sourceDirectory>${project.basedir}/src/main/resources/</sourceDirectory>
                        <outputDirectory>${project.basedir}/src/main/java</outputDirectory>
                    </configuration>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>
```

1. Search and use the latest versions
2. Right click on project > Update Maven Project
3. Right click > Run As > Maven clean
4. Right click > Run As > Maven Generate Sources

Construct Producer

1. Right click > Package > *.serializers
    1. OrderProducer.java

```
Properties props = new Properties();
props.setProperty("bootstrap.servers", "localhost:9092");
props.setProperty("key.serializer", KafkaAvroSerializer.class.getName());
props.setProperty("value.serializer", KafkaAvroSerializer.class.getName());
props.setProperty("schema.registry.url", "http://localhost:8081");

KafkaProducer<String, Order> producer = new KafkaProducer<>(props);
Order order = new Order("Bharath", "IPhone", 3);
ProducerRecord<String, Order> record = new ProducerRecord<>("OrderAvroTopic", order.getCustomerName().toString(), order);

try {
    producer.send(record);
} catch (Exception e) {
    e.printStackTrace();
} finally {
    producer.close();
}
```

Construct Consumer

1. New Consumer Project
    1. Paste all avro dependencies previously mentioned
    2. Generate stub (or copy)

Test

GenericRecord Type

Construct a GenericRecord Producer

Construct a Consumer

Schema Evolution

Schema Compatibility

Consumers in Real Time

Stop Confluent Platform

Quiz 5: Use Avro

Assignment 3: Use Avro

Section 7: More About Producers

Introduction

1. How to construct custom partitioners and put a record into whichever partition we return from custom partitioner.
2. Advanced config we can use in producer that will impact the way producer works and how Kafka broker reacts

Construct a Topic

1. cmd:

```
kafka-topics --create --zookeeper localhost:2181 --replication-factor 1 --partitions 10 --topic OrderPartitionedTopic

kafka-topics --describe --zookeeper localhost:2181 --topic OrderPartitionedTopic
```

    1. Leader is 0 - we have one broker
    2. Replicas are 0
    3. Partitions: 0 - 9

2. Editor:

    1. Producer class: com.bharath.kafka.orderproducer.customserializers.OrderProducer:

```
public class OrderProducer {
    public static void main(String[] args) {
        ...
        ProducerRecord<String, Order> record = new ProducerRecord<>("OrderPartitionedTopic", order.getCustomer...);
    }
}
```

    2. Consumer class: com.bharath.kafka.orderproducer.customdeserializers.OrderConsumer:

```
public class OrderConsumer {
    ...
    consumer.subscribe(Collections.singletonList("OrderPartitionedTopic"));
    ...
}
```

Construct Custom Partitioner

1. New class: customserializers.partitioners.VIPPartitioner which implement org.apache.kafka.clients.producer.Partitioner

```
@Override
public void configure(Map<String, ?> configs) {
    // Additional config that will help our partitioning
}

@Override
public int partition(String topic, Object key, byte[] keyBytes, Object value, byte[] valueBytes, Cluster cluster) {
    List<PartitionInfo> partitions = cluster.availablePartitionsForTopic(topic);
    if (((String) key).equals("Bharath")) {
        return 5;
    }
    return Math.abs(Utils.murmur2(keyBytes) % paritions.size() - 1); // ?
}

@Override
public void close() {
    // To close any resources that we open up during partition logic
}
```

    1. `partition` - receives key as Object as well as a byte array
        1. Receives value as object as well as a byte array
        2. `cluster` - has cluster details
            1. Partition details
            2. Number of Partitions
            3. ...
        3. `murmur2` - algorithm used to decide partition
        4. Name can be passed through configuration as well

2. Using the partitioner into OrderProducer:

```
props.setProperty("partitioner.class", VIPPartitioner.class.getName());
...
for (CustomerRecord<String, Order> record : records) {
    ...
    System.out.println("Partition: " + record.partition());
}
```

1. Run the consumer application
2. Run the producer application
    1. See console - Partition assignment is shown

Assignment 4: Construct Custom Partitioner Class

ProducerConfig Class

1. Open `OrderProducer`
    1. We don't have to hard-coding properties

```
props.setProperty(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, ...);
props.setProperty(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, ...);
props.setProperty(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, ...);
```

Acknowledgements and More

1. More properties:

```
props.setProperty(ProducerConfig.ACK_CONFIG, "0"); // "0", "1" or "all"
```

    1. Controls how partitions should receive the message before the producer considers it as a successful write/send
        1. 0 - producer will just send the message and then considers it as success
            1. It will not wait for response from the broker
                1. Cons: If leader is down, the message is lost (no re-tries)
        2. 1 - producer recieves a successful response from broker only if the leader-replica successfully receives a message
            1. Safe
        3. all - producer recieves acknowledgement from broker only once all the replicas recieve the message
            1. Pros - safest option
            2. Cons - more latency (producer has to wait for the message to be delivered to all the replicas)
    2. Buffer:

```
props.setProperty(ProducerConfig.BUFFER_MEMORY_CONFIG, "38483423");
```

        1. Producer will buffer the messages before they are sent to the broker
            1. If application is too fast in producing messages and handing them over to the API, the producers might block if the memory is not big enough
                1. Default value: 256 MB
                    1. We can configure this
    3. Compression type:

```
props.setProperty(ProducerConfig.COMPRESSION_TYPE_CONFIG, "snappy");
```

        1. By default, messages are not compressed.
            1. Only when we set this value, the messages are compressed
        2. Values:
            1. snappy - Google's
                1. Doesn't utilize much CPU
            2. gzip
                1. Compression ratio is better than snappy
                    1. CPU used is more
            3. lz4
    4. Retries:

```
props.setProperty(ProducerConfig.RETRIES_CONFIG, "2");
props.setProperty(ProducerConfig.RETRY_BACKOFF_MS_CONFIG, "400");
```

        1. Producer will retry if there is some error that can be recovered
            1. Leader has temporarily gone down
        2. Default:
            1. Producer waits for 100ms before retry
        3. Wait for 400ms before each retry
        4. Producer will not retry if there is un-recoverable exception
            1. Example: Message is too big, ...

Three More Properties

1. More properties:

```
props.setProperty(ProducerConfig.BATCH_SIZE_CONFIG, "893749233223");
```

    1. The memory size we want to allocate for a batch in bytes
    2. Default: 16kb

1. Configure it to a higher number
    1. Producer doesn't wait until the memory is filled up but hands the message over as soon as a sender thread is available to send the message to the broker
    2. If we set it to a very small value might cause problem
        1. Batch size will be very small
            1. Decreases the throughput

3. Complementing the batch size:

```
props.setProperty(ProducerConfig.LINGER_MS_CONFIG, "200");
```

    1. producer waits for 200ms before it hands over the message to a sender thread although it is available
        1. Allows us to fill the batch with few more messages
            1. Increases throughput with some compromize on latency
                1. More messages can go in a batch at once to the sender thread

4. Timeout:

```
props.setProperty(ProducerConfig.REQUEST_TIMEOUT_MS_CONFIG, "200");
```

    1. Producer will wait for 200ms for a response from the broker
        1. If it timesout, the producer will retry

2. Search for Kafka producer configuration:

    1. Confluent website
    2. Apache website

Quiz 6: More About Producers

Section 8: Message Delivery and Transactions

Introduction

1. Achieving message delivery idempotency (no duplicate deliveries)
2. Three basic message delivery semantics:
    1. At least once
    2. At most once
    3. Only Once/ idempotency
3. Advantages and disadvantages of the three semantics
4. Transactions
    1. Sending all messages or no messages at all
        1. `beginTransaction`
        2. `commitTransaction`

Message Delivery Semantics

1. Three semantics:
    1. At Least Once
    2. At Most Once
    3. Only Once / Idempotency
2. At Least Once:
    1. Steps:
        1. Producer sends a message
        2. Broker receives it
        3. Broker writes to all partitions if we have replications
        4. Once all partitions recieve the message, the broker treats it as committed
        5. Broker sends an acknowledge to the producer
    2. Issues:
        1. If broker has written to all partitions but if acknowledgement that needs to be sent is not sent
            1. Producer will retry which might result in a message duplication
                1. Same message will get re-processed
                    1. Good for use-cases if it is okay
3. At Most Once:
    1. No retries on Producer side (set to 0)
    2. Steps:
        1. Producer sends a message
        2. If broker writes to all partitions, we are good
        3. If broker fails, we have an issue
            1. The message might get lost
    3. Issues:
        1. Messages might get lost
4. Exactly Once:
    1. Producer config: `enable.idempotence` is set to `true`
    2. Steps:
        1. Producer generates a unique sequence number for every message (unique within a given partition)
        2. Broker maintains the message sequence number
        3. Broker generates a unique producerId for each producer instance
            1. The producerId uses it in the message everytime it will send it
            2. If a producer with a unique producer id is sending the same message again (it has a sequence number)
            3. Broker checks the sequence number and if it matches with a message that it has already received from the given producer, then broker will reject the message
    3. This mechanism has retries to ensure that the message is delivered at-least once, and also rejection of duplicate messages to ensure the message is delivered at-most once which makes it exactly once semantics.

1. Property to be set in Producer:

```
props.setProperty(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, "true");
```

    1. If multiple instances of producer are running, each will get a unique producer-id and if they send the same message, the duplication gets accepted.

Transactions

1. Transaction on Kafka producer is just like a DB transaction
    1. We commit all records in a transaction or we roll-back
        1. Atomicity
2. Configuration:

```
producer.initTransactions();
ProducerRecord<String, Integer> record = new ProducerRecord<>("OrderTopic", "Mac Book Pro", 10);
ProducerRecord<String, Integer> record2 = new ProducerRecord<>("OrderTopic", "Dell Laptop", 20);
try {
    producer.beginTransaction();
    producer.send(record, new OrderCallback());
    producer.send(record2);
    producer.commitTransaction();
} catch (Exception e) {
    producer.abortTransaction(); // rollback
    e.printStackTrace();
} finally {
    producer.close();
}
```

    1. If one of the records is committed but not the second one, then everything will be rolled back
        1. Producer needs to retry

Use a Transaction

1. New class: com.bharath.kafka.orderproducer.TransactionalOrderProducer

```
props.setProperty(ProducerConfig.TRANSACTIONAL_ID_CONFIG, "order-producer-1"); // id must be unique. Every producer that is starting a transact

producer.initTransactions();
```

    1. `initTransactions` - invoked before starting a transaction
        1. It ensures that any transactions initiated by previous instances of the producer with the same transactional.id are completed
        2. If previous instance had failed with a transaction in progress, it will be aborted.
        3. If the last transaction had begun completion, but not yet finished, this method awaits its completion
        4. It gets producer-id and epoch time
            1. They will be sent along every message to the broker
                1. Broker ensures idempotency using the producer-id and epoch
        5. The method will raise a TimeoutException if the transactional state cannot be initialized before expiration of `max.block.ms`

```
props.setProperty(ProducerConfig.MAX_BLOCK_MS_CONFIG, "1000"); // applicable for initTransactions, send, commitTransaction, ...
```

    2. Start transaction:

```
try {
    producer.beginTransaction();
    producer.send(record, new OrderCallback());
    producer.send(record2);
    producer.commitTransaction();
} catch (Exception e) {
    producer.abortTransaction();
    e.printStackTrace();
} finally {
    producer.close();
}
```

        1. `beginTransaction`: Called before starting every transaction
            1. `initTransactions()` must be invoked prior to invoking the method
                1. Broker must have the unique producer-id and epoch before beginning a transaction
        2. `commitTransaction`: Flushes any unset records before committing transaction
            1. If producer has more records that are buffered or batched and yet to be sent, the method ensures that the records and sent and then commits the transaction
            2. If any of the send calls hit irrecoverable error, the method will throw the last received exception immediately and the transaction will not be committed
        3. `abortTransaction` - stops any further messages from being buffered or batched to be sent

Few Pointers About Transactions

1. A producer cannot have multiple transactions open at the same time
    1. It has to close or commit a transaction before starting anothe transaction
2. Callback is not required
    1. Callback is used to just handle exceptions. However, in a transaction, we handle exception and abort transaction
3. Producers are thread safe

1. We can invoked send method from multiple threads
    1. We need to start the transaction before any threads start (?)
    2. We need to commit the transaction after all the threads finish
        1. We need to take up this responsibility

Quiz 7: Message Delivery and Transactions

Section 9: More About Consumers

Introduction

1. Topics:
    1. Consumer Groups
    2. Rebalancing
    3. How re-balancing impacts the consumer group, and partitions and offsets consumed by them
    4. How to minimize the risk of rebalancing by using offset commits
        1. Commit offsets from consumers synchronously and asynchronously
        2. How to commit a specific offset if we want to
    5. Advanced configuration on consumer side
        1. Properties that can be set that will impact the consumers and also how broker works with the consumer

Consumer Groups

1. Example: Hospital management suite of applications
    1. Components:
        1. Patient Checkout application
        2. Billing application
        3. HouseKeeping application
    2. Flow:
        1. Patient Checkout application publishes a record to Kafka Broker whenever a patient checks out
        2. Billing and HouseKeeping applications read the record, process it using their own business logic, and do whatever they are supposed to do
    3. Problem:
        1. If Patient Checkout is publishing a lot of records, they might increase the load on consumer applications and they might crash
            1. Solution: Consumer Groups
                1. Group of consumers that can read from the same topic but different partitions in the topic
2. Scaling:
    1. A topic is broken down into multiple partitions and consumer group with one or more consumers that can consume from the topic
        1. If a consumer group has a single consumer but 4 partitions, the single consumer is responsible for consuming records from all partitions.
            1. Solution: We can increase consumers in the consumer group
                1. If 4 partitions, 2 consumers:
                    1. Partition 0, 1 -> consumer 1
                    2. Partition 2, 3 -> consumer 2
                2. If 4 partitions, 4 consumers:
                    1. Each partition is read by one consumer
                3. If consumers > partitions
                    1. Additional consumers will remain idle
                        1. We need to have enough partitions to keep consumers busy
    2. Multiple consumer groups can consume from the same topic
        1. Each consumer group can have it's own copy of data
            1. Example: Billing can use consumer group 1, HouseKeeping can use consumer group 2
                1. Each consumer group will get all the data

Consumer Group Rebalancing

1. Rebalancing:
    1. If there are k partitions and k - 1 consumers in a consumer group, the k partitions will be read by the consumers
    2. If a new consumer is added to the CG, it will start consuming from a partition that was previously consumed by a previous consumer
    3. If a consumer goes downs (gracefully shut down or it has crashed), the partitions will be taken care of by other consumers in the group
    4. **Moving a partition ownership from one consumer to another consumer in a group is called consumer re-balancing**
        1. This mechanism is key to high availability and scalability of Kafka
            1. Cons: **The consumers in CG go silent during a re-balancing operation**

How Rebalancing Works

1. How consumer groups work:
    1. When a consumer group is created, Kafka allocates one of the brokers as consumer group coordinator
    2. When a new consumer joins a consumer group, it does it by sending a join request to the group coordinator
    3. The group coordinator will send all the partition details to the first consumer and makes it a consumer group leader
    4. When another consumer joins the CG by sending a join group request, the consumer group coordinator will trigger a re-balance
        1. It hands over the current partition details and consumer details to the group leader
    5. Group leader does the re-balancing
        1. It re-assign partitions to list of consumers in the CG and hands over the list to group coordinator
    6. Group coordinator will send the individual partition allocations to the other consumers
        1. Only the group coordinator and group leader will have the entire list of partitions and consumers but the other consumers will have only their list of partition allocation information
    7. The above process is repeated for each consumer that gets added after the leader
    8. When a consumer leaves the group (gracefully or due to a crash), the same process of re-balancing will take place
2. How does group coordinator know the consumer health?
    1. Consumer is responsible for sending a heartbeat
        1. If heartbeat is not received by the group coordinator for a certain amount of time (configurable), the consumer is assumed not available and re-balancing is triggered
    2. Consumer can gracefully leave the group by sending a leave request
        1. When group coordinator receives a leave request, it triggers a re-balance
3. When re-balancing happens, the consumers do nothing because group leader is re-assigning the partitions to all the consumers.
    1. **We need to minimize re-balancing or avoid it**

Offset Commits

1. Example: Assume we have 4 partitions and 4 consumers in a CG
    1. Consumers invoke poll method
    2. **Kafka doesn't keep track of which offset a consumer has reached or how many offsets a consumer has processed**
        1. **Okay if all consumers are healthy**
    3. **If a new consumer is added or if an existing consumer crashes?**
        1. Re-balancing will be triggered and partitions are re-allocated across consumers
            1. **The consumers will not know from which offset to start within a given partition**
                1. Solution: **Offset Commits**
                    1. Consumer API is allowed to do it.
                    2. Consumers can commit all the offsets they have processed to a special topic called **__consumer_offsets** (in-built topic that comes with Kafka)
                        1. Current offset is commited
                    3. If a re-balancing happens, the new consumer that is assigned the partition will look up the information in the **__consumer_offsets** topic about upto which offset and will start consuming from the next offset
                    4. Issues:
                        1. Duplicate processing: If a consumer has processed upto offset 6.
                            1. Let us assume that the consumer has polled offsets 7, 8, and 9 and will process them
                            2. It has reached offset 9 but crashes before committing
                            3. If a new consumer comes up, it will start processing from offset 7
                                1. This will result in duplicate processing
                        2. Missed Processing:
                            1. If consumer has overcommitted
                            2. If it has committed upto offset 10 but has only processed upto offset 2
                            3. If a re-balance happens, the new consumer will start processing from offset 11 missing the records from 3 to 10
2. Next:
    1. Auto commit
    2. Manual commit
        1. Synchronous
        2. Asynchronous
        3. Specific offset commit

Auto Commit

1. How does it work?
    1. OrderConsumer

```
props.setProperty("enable.auto.commit", "false"); // default is "true"
```

        1. Auto-commit time interval is 5 s
            1. Poll method drives it
                1. When it is invoked for the first time
                    1. It will fetch and offset
                    2. It will start a timer for the auto-commit
                    3. The records will be processed
                2. Next time the `poll` method is invoked, the poll will check if the default 5 seconds have elapsed.
                    1. Only if the 5 seconds have elapsed, it will commit the previous offset
                    2. If 5 seconds have not elapsed, it will fetch the next offset of records and consumer will process them
                    3. Next time poll is invoked, if the timer has gone beyond 5 second mark, it will commit both the previous offsets
                        1. The timer will be reset to 0
    2. Configuring time limit:

```
props.setProperty("auto.commit.interval.ms", "7000"); // 7 seconds
```

    3. `close` also does a commit
2. Problem with auto-commit:
    1. Re-balancing:
        1. Let us assume the default auto-commit time to be 5 seconds
        2. Let us say 2 s has elapsed
        3. Let us say the consumer has processed 400 records
        4. If a rebalance happens
            1. New consumer will process from latest offset
                1. Since 1-400 are not committed, they will be consumed again
                    1. Duplicate processing will happen
        5. Solution:
            1. We can reduce the timer but the problem is not entirely eliminated

Sync Commit

1. To gain more control on when the offsets will be committed, we can set the following property:

```
// props.setProperty("auto.commit.interval.ms", "2000");
props.setProperty("auto.commit.offset", "false"); // not committed automatically
```

2. We can finish processing and commit:

```
while (true) {
    ConsumerRecords<String, Order> records = consumer.poll(Duration.ofSeconds(20));
    for (ConsumerRecord<String, Order> record : records) {
        String customerName = record.key();
        Order order = record.value();
        System.out.println("Customer Name: " + customerName);
        System.out.println("Product: " + order.getProduct());
```

```
            System.out.println("Quantity: " + order.getQuantity());
            System.out.println("Partition: " + record.partition());
        }
        consumer.commitSync(); // this will commit the entire commit offset
    } finally {
        consumer.close();
    }
```

1. `consumer.commitSync()`: It will commit the entire commit offset
    1. It also does retries
        1. It will retry if there are any exceptions
            1. Unless it is an un-recoverable exception
    2. We still have re-balancing problem
        1. Let us assume we have 1000 records and 400 records are processed.
            1. If re-balancing happens, the new consumer will process 1-400 records (from the beginning)
                1. **Duplication problem**
    3. Pros: We commit as soon as the offset is processed and not after sometime (as in auto-commit)

Async Commit

1. `commitSync` - blocks consumer application until the broker responds back to the commit request (degrades performance)
    1. Retries if commit fails
2. `commitAsync` - commit request will be sent to the broker and the next `poll` request is executed, next offset is processed while the current commit is in progress.

    1. Challenges:

        1. Doesn't retry if a commit fails for some reason
            1. Reason:
                1. Let us assume that when the poll is invoked for the first time, we have received offset from records 1-1000
                2. The records are processed
                3. The `commitAsync` method is invoked
                4. The `poll` will be invoked again immediately
                5. Let us assume the previous commit has failed
                6. Let us assume that the records from 1001-2000 have been processed successfully and committed successfully
                7. If a retry is invoked, the order will get jumbled up
                    1. If a re-balance is invoked at this point, the new consumer will re-process all the records starting from 1
                        1. Duplicate processing

    2. Supports a callback:

```
consumer.commitAsync(new OffsetCommitCallback() {
    @Override
    public void onComplete(Map<TopicPartition, OffsetAndMetadata> offsets, Exception exception) {
         if (exception != null) {
            System.out.println("Commit failed for offset " + offsets);
         }
    }
});
```

        1. Whenever a commit finishes the callback is invoked and we can take appropriate action
            1. We can also retry here if we want
                1. We need to take into consideration, the problem with retry
            2. If there is any exception during commit, the exception is also passed as an argument
            3. We get offsets that were processed
                1. It is a map from `TopicPartition` to `OffsetAndMetadata`

Commit Custom Offset

1. Commit sync and async methods will commit the entire offset returned by the `poll` method
    1. We need to wait for all the messages in the offset to be processed before we commit
        1. If a re-balance happens between processing of the records then the next consumer which takes up the partition will start processing again from the beginning of the offset
            1. Solution: Custom commits
                1. We can do more frequent commits
2. Both `commitSync` and `commitAsync` have overloaded methods

    1. Say, we want commit to happen for every 10 records that are processed

```
int count = 0;
for (...) {
    ...
    if (count % 10 == 0) {
        consumer.commitAsync(Collections.singletonMap(new TopicPartition(record.topic(), record.partition()), new OffsetAndMetadata(recorc
            ...
        });
    }
}
```

        1. `record.offset() + 1` - next consumer will start from the next record
        2. The first argument:
            1. Map of topic-partition to offset that needs to be committed
                1. Key: TopicPartition(topic, partition) object
                2. Value: OffsetAndMetadata(offset) object

Construct a RebalanceListener

1. How to use consumer-rebalance listener to take it a step further

    1. It is a special class with methods

1. The methods will be invoked when a re-balance happens

2. Overloaded version of `subscribe`

```
class RebalanceHandler implements ConsumerRebalanceListener {
    @Override
    public void onPartitionsRevoked(Collection<TopicPartition> partitions) {

    }

    @Override
    public void onPartitionsAssigned(Collection<Partition> partitions) {

    }
}

consumer.subscribe(Collections.singletonList("OrderPartitionedTopic", new RebalanceHandler());
```

1. `onPartitionsRevoked` - invoked when a re-balance is triggered and before the partitions are being revoked from this consumer
    1. If there is any cleanup on consumer side and if there are any records which are not committed yet, we can commit those offsets here
        1. Last chance before consumer loses the partition completely
2. `onPartitionsAssigned` - invoked when partitions are assigned
    1. When new partitions are assigned we can take necessary action in this method

Commit Last Offset Processed

1. We will commit offsets that are processed but yet to be committed in re-balance handler

```
consumer.commitSync();
```

1. Extract map:

```
Map<TopicPartition, OffsetAndMetadata> currentOffsets = new HashMap<>();

KafkaConsumer<String, Order> consumer = new KafkaConsumer<>();
...
onPartitionsRevoked(...) {
    consumer.commitSync(currentOffsets);
}
...
currentOffsets.put(new TopicPartition(record.topic(), record.partition()), new OffsetAndMetadata(record.offset() + 1)); // has latest offs

if (count % 10 == 0) {
    ...
}
```

Using ConsumerConfig Class

1. Consumer configuration:

```
public class OrderConsumer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.setProperty(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
        props.setProperty(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, "org.apache.kafka.common.);
        props.setProperty(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, "...");
        props.setProperty(ConsumerConfig.GROUP_ID_CONFIG, "OrderGroup");

        ...
    }
}
```

Min Fetch Size and Timeouts

1. 4 important poperties:

```
props.setProperty(ConsumerConfig.FETCH_MIN_BYTES_CONFIG, "1024"); // Kafka broker will wait until the mentioned amount of data exists to be sen
props.setProperty(ConsumerConfig.FETCH_MAX_WAIT_MS_CONFIG, "200"); // Kafka broker will wait for 200 ms before it sends the data. Which ever oc

props.setProperty(ConsumerConfig.HEARTBEAT_INTERVAL_MS_CONFIG, "1000"); // For every 1000 ms, the consumer has to send a heartbeat to the coord

props.setProperty(ConsumerConfig.SESSION_TIMEOUT_MS_CONFIG, "3000"); // Tells the broker, for how long can the consumer go without sending the
```

1. Good practice is: heartbeat interval must be 1/3rd the session timeout config

Four More Properties

1. 4 more properties:

```
props.setProperty(ConsumerConfig.MAX_PARTITIONS_FETCH_BYTES_CONFIG, "1MB"); // Default: 1MB, controls the max number of bytes the server return

props.setProperty(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "latest");

props.setProperty(ConsumerConfig.CLIENT_ID_CONFIG, "OrderConsumer");

props.setProperty(ConsumerConfig.MAX_POLL_RECORDS_CONFIG, "100");
```

1. ConsumerConfig.MAX_PARTITIONS_FETCH_BYTES_CONFIG: If we have 30 partitions, 5 consumers, each consumer gets 6 partitions
    1. We need to ensure that each consumer has at-least 6 MB space
        1. However, it is better to give more - 12 MB
            1. If one of the consumers goes down, another consumer needs to handle another partition
    2. The property says how many bytes per partition the broker sends to the consumer

2. ConsumerConfig.AUTO_OFFSET_RESET_CONFIG: Controls consumer behavior if it starts reading a partition if it doesn't have a committed offset
    1. "lastest" - consumer will start reading those records which came to the partition after the consumer started running
        1. Ignores previous records
    2. "earliest" - consumer will start reading from the beginning of the partition
        1. It will process all the records from the beginning of the partition
3. ConsumerConfig.CLIENT_ID_CONFIG: Used by broker for logging, metrics, and any quota allocation purposes
4. ConsumerConfig.MAX_POLL_RECORDS_CONFIG: The maximum number of records the poll can return
    1. Controls the amount of data our application needs to process in the polling loop
        1. Each time `poll` is called, the specified number of records will be returned

Partition Assignor Strategy

1. Another property:

```
props.setProperty(ConsumerConfig.PARTITION_ASSIGNMENT_STRATEGY_CONFIG, RangeAssignor.class.getName());
```

    1. Internally, a partition assigner is responsible to assign partitions to consumers in consumer-group
    2. Two partition assigners:
        1. `RangeAssignor`: Default
            1. Assume there are two topics and three partitions each
            2. Assume there are two consumers in the consumer group
            3. If `RangeAssignor` is used:
                1. It will take consecutive subset of partitions from each topic and assigns them to consumer
                    1. Consumer 1:
                        1. P0, P1 from Topic 1
                        2. P0, P1 from Topic 2
                    2. Consumer 2:
                        1. P2 from Topic 1
                        2. P2 from Topic 2
                2. It doesn't equally divide
                    1. The first consumer gets more partitions than the second consumer
        2. `RoundRobinAssignor`:
            1. It will combine the partitions of both the topics and assign in a round robbin fasion
                1. Consumer 1:
                    1. P0 from Topic 1
                    2. P2 from Topic 1
                    3. P1 from Topic 2
                2. Consumer 2:
                    1. P1 from Topic 1
                    2. P0 from Topic 2
                    3. P2 from Topic 2
            2. Consumers mostly get equal number of partitions
                1. If not one of them will get one extra (if two consumers)

Quiz 8: More About Consumers

Section 10: Construct a Simple Consumer

Introduction

1. Kafka allows us to have a standalone consumer which doesn't have to be a part of a consumer-group (rare)
    1. There is no re-balancing, group-coordinator, ...
    2. The consumer is responsible to take care of partitions

        1. **We need to assign partitions**

```
List<PartitionInfo> partitionInfo = consumer.partitionsFor("SimpleConsumerTopic");

List<TopicPartition> partitions = new ArrayList<>();

for (PartitionInfo info : partitionInfos) {
    System.out.println(info.partition());
    partitions.add(new TopicPartition("SimpleConsumerTopic", info.parition());
}

consumer.assign(partitions); // instead of subscribe
```

            1. We can also assign specific partitions

        2. If new partitions are created, we need to invoke `partitionsFor` method again

            1. We might need to loop and check

Construct a Simple Consumer

1. New class: `com.bharath.kafka.simpleconsumer.SimpleConsumer`

```
public class SimpleConsumer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.setProperty(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
        props.setProperty(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, "...");
        props.setProperty(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, "...");

        KafkaConsumer<String, Integer> consumer = new KafkaConsumer<>(props);
        List<TopicPartition> partitions = new ArrayList<>();
        List<PartitionInfo> partitionInfos = consumer.partitionsFor("SimpleConsumerTopic");
```

```
            for (PartitionInfo info : partitionInfos) {
                partitions.add(new TopicPartition("SimpleConsumerTopic", info.partition()); // The topic is automatically created when we run the p
            }

            consumer.assign(partitions);

            ConsumerRecords<...>
            ...
        }
    }
```

Construct a Producer and Test

1. Simple producer and test:
2. New class: `com.bharath.kafka.orderproducer.SimpleConsumerTestProducer`

```
public class SimpleConsumerTestProducer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.setProperty(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
        props.setProperty(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, "...");
        props.setProperty(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, "...");

        KafkaProducer<String, Integer> producer = new KafkaProducer<String, Integer>(props);
        ProducerRecord<String, Integer> record = new ProducerRecord<>("SimpleConsumerTopic", "Mac Book");

        try {
            producer.send(record, new OrderCallback());
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            producer.close();
        }
    }
}
```

    1. Run Producer which instantiates the topic
    2. Set another property for consumer:

```
props.setProperty(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");
```

        1. Otherwise, it will start consuming only those messages that arrived after it starts

    3. Run the consumer

Few Important Points

1. Things to remember while working with simple consumer:
    1. `poll` method gets offset of records, consumer processes them, and auto-commit is triggered by default

```
// after records are processed
consumer.commitAsync();
```

        1. Throws an exception:

```
org.apache.kafka.common.errors.InvalidGroupIdException: To use the group management or offset commit APIs, you must provide a valid
```

            1. To use offset commit API, we need to part of a valid group id even if we don't want consumer group
                1. Solution:

```
props.setProperty(ConsumerConfig.GROUP_ID_CONFIG, "SimpleConsumerGroup"); // unique value
```

            2. We must not add a simple consumer to an active group (commit exception)
            3. Same consumer group can be used for multiple simple consumers

Section 11: Stream Processing

Introduction

1. Topic:
    1. What is real-time stream processing
2. Use-Case: Big data
    1. Data is first stored in systems we call data lakes
    2. We then use tools to pull the data, analyze it and generate reports that are used to make business decisions
    3. The process is called **near real-time**
        1. There is a slight time delay between data generation, data being pulled and analysis being done and report getting generated
3. Data stream:
    1. It is a continuous flow of business events or data
    2. Unlike in big-data, we do not store the data in database or datalake but it is data in-motion
    3. It doesn't have a pre-determined beginning or ending
    4. As the data flows through the stream processing applications, the applications can apply computational logic (aggregations, transformations, ...) on the data and the reports will be generated in real-time
    5. Real-time stream processing applies to:
        1. Data platforms, micro-services

        1. Microservices applications can use the Kafka real-time processing along with other big-data tools like Storm, Spark, ... to analyse the data right away and generate the reports.
    2. Monitoring trucks, cars
        1. Real-time stream processing can be used to monitor the trucks, cars, ... to see that they are on-time to the destination and if something goes wrong, we can fix it right away
    3. Retail, Hotel and Travel
        1. For recommendations, ...
    4. Factories
        1. We can process data from IoT devices and make intelligent decisions
    5. Hospitals
        1. To monitor the health of a patient and take intelligent decisions
    6. Financial Transactions
        1. Real-time data processing can be used for payment transactions and stock transactions, ...
6. Kafka makes is easy for us to construct the computational logic as data flows by giving us a Kafka streams library
    1. Kafka Stream Library:
        1. StreamsDSL
            1. This is on top of Processor API
                1. DSL - Domain Specific Langauge
                2. It is simple to use as data flows through the applications
                    1. For joins
                    2. For transformations
        2. Processor API
            1. We can use this if StreamsDSL is not good enough

Usecase and API

1. Kafka Streams Library:

```
<dependency>
    <groupId>org.apache.kafka</groupId>
    <artifactId>kafka-streams</artifactId>
    <version>2.8.0</version>
</dependency>
```

    1. It is an abstraction built into the Kafka client library
        1. It transitively pulls it and uses it

2. Data-flow use-case:

    1. The data flows from an input topic to an output topic
    2. We can apply transformational logic in-between

3. Steps:

    1. Set required properties

```
Properties props = new Properties();
props.put(StreamsConfig.APPLICATION_ID_CONFIG, "streams-dataflow");
props.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
props.put(StreamsConfig.DEFAULT_KEY_SERDE_CLASS_CONFIG, Serdes.String()).getClass().getName());
props.put(StreamsConfig.DEFAULT_VALUE_SERDE_CLASS_CONFIG, Serdes.String().getClass().getName());
```

        1. `StreamsConfig` - properties are defined in this class as constants
        2. `APPLICATION_ID_CONFIG` - unique id that we need to provide for every streaming application
        3. `BOOTSTRAP_SERVERS_CONFIG` - broker details
        4. `DEFAULT_KEY_SERDE_CLASS_CONFIG` - Key for serializer and deserializer (SERDE)
            1. A streaming application is both a consumer and a producer
        5. `DEFAULT_VALUE_SERDE_CLASS_CONFIG` - Value for serializer and deserializer
        6. `Serdes` - factory class
            1. It has methods when invoked will return the appropriate SERDE class
            2. `Serdes.String()` - It will return serializer and deserializer required to handle `String` data
                1. Other options:
                    1. `Serdes.Long()`
                    2. `Serdes.Double()`
                    3. ...
                2. `String()` - wrapper around `StringSerializer` and `StringDeserializer` classes

    2. Define computational logic

```
StreamsBuilder builder = new StreamsBuilder();
KStream<String, String> stream = builder.stream("streams-dataflow-input");
stream.foreach((key, value) -> System.out.println("Key and Value " + key + " " + value));
stream.filter((key, value) -> value.contains("token"))
    // .mapValues(value -> value.toUpperCase())
    .map((key, value) -> KeyValue.pair(key, value.toUpperCase()))
    .to("streams-dataflow-output");

Topology topology = builder.build();
System.out.println(topology.describe());
```

        1. Topology for the application
            1. `StreamsBuilder` - used to construct a topology (topology builder)
            2. `builder.stream` - used to stream data from an input topic
            3. `stream` - object used to apply transformations on data
                1. It has methods used to transform data
            4. `to` - points to output topic
            5. `builder.build()` - used to build a topology object

    3. Construct a Kafka Streams client object

```
KafkaStreams streams = new KafkaStreams(topology, props);
streams.start();
```

1. `streams.start()` - only when we run this, the data will start flowing

4. Shutdown the stream

```
Runtime.getRuntime().addShutdownHook(new Thread(streams::close));
```

1. When the program is running, the `start` will run it in an infinite loop
2. When we stop the program (ctrl + c), the stream will stop
    1. The shutdown hook is called when the program is closed.
        1. `Runtime.getRuntime()` - This will return the runtime of the stream program
        2. `addShutdownHook` - A new thread is passed to this method which calls `streams::close` method

Construct Topics

1. Setup of topics required for simple data-flow streaming application

2. Commands:

```
kafka-topics --create \
    --bootstrap-server localhost:9092 \
    --replication-factor 1 \
    --partitions 1 \
    --topic streams-dataflow-input

kafka-topics --create \
    --bootstrap-server localhost:9092 \
    --replication-factor 1 \
    --partitions 1 \
    --topic streams-dataflow-output

kafka-console-producer --bootstrap-server localhost:9092 --topic streams-dataflow-input

kafka-console-consumer --bootstrap-server localhost:9092 \
    --topic streams-dataflow-output \
    --property print.key=true \
    --property print.value=true \
    --property key.deserializer=org.apache.kafka.common.serialization.StringDeserializer \
    --property value.deserializer=org.apache.kafka.common.serialization.StringDeserializer
```

1. Confluent Kafka:

```
confluent kafka topic create streams-dataflow-input --partitions 1

confluent kafka topic create streams-dataflow-output --partitions 1
```

Construct Project

1. New maven project:
    1. maven-archetype-quickstart
    2. groupId: com.bharath.kafka
    3. artifactId: streams-demo
    4. java version: 14
    5. Remove plugins section
2. Open web browser:

    1. search for kafka streams maven dependency

    2. copy

    3. Add it:

```
<!-- https://mvnrepository.com/artifact/org.apache.kafka/kafka-streams -->
<dependency>
    <groupId>org.apache.kafka</groupId>
    <artifactId>kafka-streams</artifactId>
    <version>3.5.1</version>
</dependency>
```

    4. Right click

        1. Maven > Update project

    5. dependencies:

        1. kafka-client....
        2. Other dependencies

Step 1 - Configure Properties

1. New class: DataFlowStream

```
Properties props = new Properties();
props.put(StreamsConfig.APPLICATION_ID_CONFIG, "streams-dataflow");
props.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");

// Confluent Kafka Related Config
//props.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG, "<SERVER_DOMAIN_NAME>:9092");

props.put(StreamsConfig.DEFAULT_KEY_SERDE_CLASS_CONFIG, Serdes.String().getClass().getName());
```

```
props.put(StreamsConfig.DEFAULT_VALUE_SERDE_CLASS_CONFIG, Serdes.String().getClass().getName());

// Confluent Kafka Related Config
props.put(StreamsConfig.SECURITY_PROTOCOL_CONFIG, "SASL_SSL");
props.put("sasl.jaas.config", "org.apache.kafka.common.security.plain.PlainLoginModule required username='<API_KEY>' password='<API_SECRET>';")
props.put("sasl.mechanism", "PLAIN");
```

1. APPLICATION_ID_CONFIG - every streaming application should have a unique id
    1. When we run multiple instances of the same application, they should have the same id
2. BOOTSTRAP_SERVERS_CONFIG - supports multiple servers as comma-separated values
3. DEFAULT_KEY_SERDE_CLASS_CONFIG - Default which can be overridden

Step 2 - Construct Topology

1. Topology code:

```
StreamsBuilder builder = new StreamsBuilder();
KStream<String, String> stream = builder.stream("streams-dataflow-input");
stream.foreach((key, value) -> System.out.println("Key and Value " + key + " " + value));

Topology topology = builder.build();
```

1. `StreamsBuilder` - topology builder
2. `KStream` - Kafka-Stream object
    1. `String` - key
    2. `String` - value
3. Each record is handed over to the lambda
4. `build` - builds a topology

Step 3 - Start and Stop Stream

1. Start and stop the stream

```
KafkaStreams streams = new KafkaStreams(topology, props);
streams.start();

Runtime.getRuntime().addShutdownHook(new Thread(streams::close));
```

1. `KafkaStreams` - streams client object
2. `start` - starts the stream
3. Right click and run the program
    1. It runs indefinitely
4. Resource leak. The streams is not closed
5. `Runtime.getRuntime().addShutdownHook(new Thread(streams::close))` - closes the stream on shutdown

Test

1. Right click and start the application
2. Run the following command in console:

```
kafka-console-producer --bootstrap-server localhost:9092 --topic streams-dataflow-input
```

1. Confluent Kafka

```
confluent kafka topic produce streams-dataflow-input
```

3. Type data to send:

```
Kafka streams is easy and awesome
```

1. The output will be shown on the console in the IDE

Describe Topology

1. Inspecting topology before running the stream:
    1. Used to ensure that our computational logic is okay.
2. Code:

```
System.out.println(topology.describe());
...
//streams.start();
```

1. Describes the entire topology

```
Topologies:
    Sub-topology: 0
        Source: KSTREAM-SOURCE-0000...0 (topics: [streams-dataflow-input])
            --> KSTREAM-FOREACH-0000...1
        Processor: KSTREAM-FOREACH-0000...1 (stores: [])
            --> none
            <-- KSTREAM-SOURCE-0000...0
```

1. Terminating transformations do not return anything

2. Arrows indicate data flow

```
Topic
|Consume
v
Source Processor
|KStream
v
ForEach Processor
```

3. Uncomment `start()`

Write to Output Topic

1. Code:

```
...
stream.to("streams-dataflow-output");
```

    1. Expects the topic to be created before running the application

2. Output topic creation:

```
kafka-console-consumer --bootstrap-server localhost:9092 \
    --topic streams-dataflow-output \
    --property print.key=true \
    --property print.value=true \
    --property key.deserializer=org.apache.kafka.common.serialization.StringDeserializer \
    --property value.deserializer=org.apache.kafka.common.serialization.StringDeserializer
```

    1. Confluent Kafka:

```
confluent kafka topic create --partitions 1 --if-not-exists streams-dataflow-output
```

    2. Enter data in console

        1. Data is flows to the output topic

    3. Topology:

```
Sink node
```

        1. Data is flowing to both Processor node and Sink node

Use Filter Method

1. Code:

```
KStream<String, String> filteredStream = stream.filter((key, value) -> value.contains("token"));
filteredStream.to("streams-dataflow-output");

// or

stream.filter((key, value) -> value.contains("token")).to("streams-dataflow-output");
```

    1. Takes a predicate, filters out records using the predicate and constructs a new stream which has only the filtered records / messages

    2. Run the application:

```
filter works only with token
filter does not work
```

        1. Second message doesn't pass through

Use Map Methods

1. Code:

```
stream.filter(...).
    .mapValues(value -> value.toUpperCase())
    .to(...);
```

    1. `mapValues` - transforms value into a different value based on value-mapper logic we pass
        1. Transformation can be sophisticated

2. Code:

```
stream.map((key, value) -> new KeyValue<>(key, value.toUpperCase()))
```

    1. `map` - can be used to transform both key and value

    2. Value can be a key:

```
stream.map((key, value) -> new KeyValue<>(value, value.toUpperCase())
```

    3. Another way:

```
stream.map((key, value) -> KeyValue.pair(key, value.toUpperCase()))
```

Assignment 5: Streaming Basics

1. Filter integers, filter out odd numbers, and triple them. Write the output to console

```
stream.filter((key, value) -> (value % 2) != 0)
    .mapValues(value -> 3 * value)
    .to("streams-dataflow-output");
```

Word Count Usecase

1. As messages flow from input topic to the output topic, we can have a topology that calculates the number of times a word appears

KTable

1. Code:

```
KGroupStream<String, String> kGroupedStream = stream.flatMapValues(value -> Arrays.asList(value.to)).groupBy((key, value) -> value);
KTable<String, Long> countsTable = kGroupedStream.count();
countsTable.toStream().to("streams-wordcount-output", Produced.with(Serdes.String(), Serdes.Long());
```

    1. `count` - aggregate function

2. KTable:

    1. It is a representation of an update stream (change-log)
        1. As the records flow through the stream, it maintains updates for the records with the same key
            1. A stateful component
    2. The table is stored in a local store that is created automatically
        1. As the records flow through the stream, the count of the words from previous messages
    3. Handles upsert (update | insert) operation
        1. If we want to delete, we can take a key and pass null as the value

Construct Topics

1. Topics for word-count demo application

2. Commands:

```
kafka-topics --create \
    --bootstrap-server localhost:9092 \
    --replication-factor 1 \
    --partitions 1 \
    --topic streams-wordcount-input

kafka-topics --create \
    --bootstrap-server localhost:9092 \
    --replication-factor 1 \
    --partitions 1 \
    --topic streams-wordcount-output
```

Implement Word Count

1. Make a copy of `DataFlowStream` and name it `WordCountStream`

```
KStream<String, String> stream = builder.stream("streams-wordcount-input");

stream.flatMapValues(value -> Arrays.asList(value.toLowerCase().split(" "));

...
```

    1. `flatMapValues` - returns multiple records
        1. Returns a list of values split by space character with key as `null`

Aggregate Using groupBy and Count

1. SQL analogy:

```
select count(value) kstream group by value;
```

    1. It is more tricky here:

```
KGroupedStream<String, String> kGroupedStream =  stream.flatMapValue(...).
    .groupBy((key, value) -> value); // sets value as the key
KTable<String, Long> countsTable = KGroupedStream.count(); // returns a KTable
countsTable.toStream().to("streams-wordcount-output", Produced.with(Serdes.String(), Serdes.Long()));
```

        1. Converted to stream and sent to output topic
        2. `Produced.with` - takes serializer and deserializer for both key and value (because value is `Long`)

Test

1. Property needed:
   props.put(StreamsConfig.CACHE_MAX_BYTES_BUFFERING_CONFIG, 0);
    1. Required for `KTable`
        1. It might take sometime before the contents of KTable are streamed
        2. By default, the value is 1
            1. It does some buffering and only if there is lot of data and counting, then it will push it to the Stream
                1. Production has a lot of data so this prop is not required

2. Start the stream:
    1. Produce data:

```
kafka-console-producer --bootstrap-server localhost:9092 --topic streams-wordcount-input

kafka-console-consumer --bootstrap-server localhost:9092 \
    --topic streams-wordcount-output \
    --from-beginning \
    --property print.key=true \
    --property print.value=true \
    --property key.deserializer=org.apache.kafka.common.serialization.StringDeserializer \
    value.deserializer=org.apache.kafka.common.serialization.LongDeserializer
```

        1. Run
    2. Send test data

Quiz 9: Stream Processing

Section 12: Spring Boot and Kafka

Introduction

1. We wrote a lot of boilerplate code and we repeat it for all producers and consumers
    1. Solutions: `spring-kafka`
        1. It is an abstraction on top of Kafka clients library
        2. It makes it simple to construct producers and consumers
2. Producer side:
    1. `KafkaTemplate` is injected
        1. Boilerplate code will be gone
3. Consumer side:
    1. `@KafkaListener` annotation
        1. Listener Container is maintained internally
        2. A method is marked with `@KafkaListener` annotation

API Walkthrough

1. New Spring Boot Project
    1. Producer Project
        1. `spring-boot-starter-web`
        2. `spring-kafka` dependency
        3. RESTful endpoint will receive user's data
            1. The endpoint will write data to a Kafka topic
            2. Data: user's name and age
        4. Data is sent using `KafkaTemplate` instance
        5. The configuration is in **application.properties**
    2. Consumer Project
        1. Add the above dependencies
        2. `@Service` class with a method marked with `@KafkaListener` annotation
        3. Recieves data and processes it
        4. The configuration is in **application.properties**

Construct Producer Project

1. New Spring Boot project:
    1. New starter Project
        1. name: user-producer
        2. group: com.bharath.kafka
        3. Description: User producer
        4. Next:
            1. Search for kafka
                1. Spring for Apache Kafka
            2. Search for web
                1. Spring Web
    2. Open pom.xml
        1. `spring-kafka`
        2. `spring-kafka-test`
            1. Allows us to write kafka specific tests

Construct the Producer Service

1. New class: com.bharath.kafka.service.UserProducerService

```
@Service
public class UserProducerService {
    @Autowired
    private KafkaTemplate<String, Integer> kafkaTemplate;

    public void sendUserData(String name, int age) {
        kafkaTemplate.send("user-topic", name, age);
    }
}
```

    1. `send` overloaded:
        1. Topic
        2. data

3. Key
4. data

Construct the REST Endpoint

1. New class: com.bharath.kafka.controllers.UserController

```
@RestController
@RequestMapping("/userapi")
public class UserController {
    @Autowired
    private UserProducerService service;

    @PostMapping("/publishUserData/{name}/{age}")
    public void sendUserData(@PathVariable("name") String name, @PathVariable("age") int age) {
        service.sendUserData(name, age);
    }
}
```

Construct the Consumer

1. New Spring Starter project:
    1. Name: user-consumer
    2. group: com.bharath.kafka
    3. Description: User consumer
    4. Next:
        1. Search for kafka
            1. Spring for Apache Kafka
        2. Search for web
            1. Spring Web
2. New class: com.bharath.kafka.service.UserConsumerService

```
@Service
public class UserConsumerService {

    @KafkaListener(topics = {"user-topic"})
    public void consumerUserData(int age) {
        System.out.println("User's age is: " + age);
    }
}
```

Configure

1. Open application.properties under Producer project

```
spring.kafka.producer.bootstrap-servers = localhost:9092
spring.kafka.producer.key-serializer = org.apache.kafka.common.serialization.StringSerializer
spring.kafka.producer.value-serilizer = org.apache.kafka.common.serialization.IntegerSerializer
spring.kafka.properties.security.protocol=SASL_SSL
spring.kafka.properties.sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required username='<api-key>' password='<api-s
spring.kafka.properties.sasl.mechanism=PLAIN
```

    1. cmd + shift + t or ctrl + shift + t to open properties values autofill
        1. Copy the package name
2. Open application.properties under Consumer project
    1. Copy paste the above properties

```
spring.kafka.consumer.bootstrap-servers = localhost:9092
spring.kafka.consumer.key-deserializer = org.apache.kafka.common.serialization.StringDeserializer
spring.kafka.producer.value-deserilizer = org.apache.kafka.common.serialization.IntegerDeserializer
spring.kafka.consumer.group-id = user-group
spring.kafka.properties.security.protocol=SASL_SSL
spring.kafka.properties.sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required username='<api-key>' password='<
spring.kafka.properties.sasl.mechanism=PLAIN
```

3. Topic creation:

```
confluent kafka topic create user-topic --partitions 1
```

Test

1. Right click and run as Spring Boot App
    1. Producer
    2. Consumer:
        1. Open application.properties

```
server.port = 8085
```

2. Open PostMan
    1. POST: http://localhost:8080/userapi/publishUserData/bob/40

Use Object Type

1. How to produce custom objects

2. New pojo class: .kafka.dto.User

```
public class User {
    private String name;
    private int age;
    private String favGenre;

    // Getters and setters
}
```

3. Controller:

```
@PostMapping("/publishUserData")
public void sendUserData(@RequestBody User user) {
    service.send(user);
}
```

4. Service:

```
@Autowired
private KafkaTemplate<String, User> kafkaTemplate;

public void sendUserData(User user) {
    kafkaTemplate.send("user-topic", user.name, user);
}
```

5. Configure serializer: application.properties:

```
spring.kafka.producer.value-serializer = org.springframework.kafka.support.serializer.JsonSerializer
```

Configure JsonDeserailizer

1. Consumer:
    1. Copy DTO package: (can be in a common project)

```
@KafkaListener(topics = {"user-topic"})
public void consumerUserData(User user) {
    System.out.println("Users Age Is: " + user.getAge() + " Fav Genre " + user.getFavGenre());
}
```

    2. application.properties:

```
spring.kafka.consumer.value-deserilizer = org.springframework.kafka.support.serializer.JsonDeserializer
spring.kafka.consumer.properties.spring.json.trusted.packages = com.bharath.kafka.dto
...
```

        1. There will be trust issues
            1. JsonDeserailizer does not trust all DTOs
                1. We need to give the packages that JsonDeserailizer can trust
                    1. We can give one or more packages
                    2.
                        -
                            - trust all packages

Test

1. Run Producer as Spring Boot app
2. Run Consumer as Spring Boot app
3. Go to postman:

```
POST: http://localhost:8080/userapi/publishUserData

{
    "name": "Bharath",
    "age": 40,
    "favGenre": "thriller"
}
```

4. The data appears in Consumer console

Quiz 10: Spring Boot and Kafka

Assignment 6: Spring Boot and Kafka

1. CreditCard Producer project, Credit Card Consumer project
2. CreditCard DTO:
    1. fullName
    2. cardNumber
    3. expDate
    4. secCode
3. REST endpoint
4. Consumer will recieve the message and prints the information

Section 13: Wrap Up

Quiz 11: Final Quiz

Bonus Lecture

1. Courses:
    1. https://www.udemy.com/course/kafka-fundamentals-for-java-developers/learn/lecture/27907362#content